

PROGETTAZIONE DI SISTEMI DIGITALI

Michele De Fusco, Luigi Lo Bianco, Stefano Scarcelli, Costanza Ferrari

Relazione del progetto

Indice

1	Introduzione	3
2	Componenti Base	5
2.1	Full Adder	5
2.1.1	Logica Implementativa	5
2.2	N-Bit Ripple Carry Adder (RCA)	6
2.2.1	Architettura Parametrica	6
2.3	Strategie di Testing	7
2.3.1	Unit Testing: Full Adder (Exhaustive)	7
2.3.2	Unit Testing: Ripple Carry Adder (Directed)	8
3	Il moltiplicatore	9
3.1	Implementazione	9
3.1.1	Alcune note sui prodotti	12
3.1.2	Un esempio	12
3.2	Il circuito moltiplicativo	13
3.3	Il testing	14
4	Il Carry Save Adder Tree	17
4.1	Implementazione	17
4.2	Il Carry Select Adder	22
4.3	Il circuito finale	24
4.4	Il testing	25
4.4.1	Caso positivo	26
4.4.2	Caso negativo	27
5	BufferLine	29
5.1	Buffer e Pipeline: L'architettura Sliding Window	29

5.1.1	Architettura del Dataflow	29
5.1.2	Implementazione degli Shift Register Arrays	30
5.1.3	Gestione del Flush e Padding	31
6	FSM ed AXI-Stream	32
6.1	Implementazione della FSM e Interfacce AXI-Stream	32
6.1.1	Architettura della FSM	32
6.1.2	Gestione del Protocollo AXI4-Stream	33
6.1.3	Sincronizzazione e Rimozione dello stato IDLE	34
6.1.4	Controllo del Dataflow e Backpressure	35
6.2	Verifica Funzionale: Integrazione FSM e Buffer Line	35
6.2.1	Setup di Simulazione	36
6.2.2	Generazione degli Stimoli e Handshake AXI	36
6.2.3	Esiti della Verifica	38

Capitolo 1

Introduzione

La traccia richiedeva l'implementazione di un circuito per il filtraggio di un'immagine greyscale di almeno 32x32 pixel rappresentati su 8 bit unsigned.

Il filtro deve essere composto da 3x3 coefficienti (scelti in modo arbitrario dal gruppo di lavoro) rappresentati su 4 bit in complemento a 2.

La finestra di convoluzione ha la stessa dimensione del filtro, quindi 3x3 e scorre lungo tutta l'immagine per filtrare un pixel alla volta; il procedimento di invio dei dati e riempimento del buffer line è gestito dalla state machine attraverso alcuni segnali che verranno descritti successivamente.

L'operazione di filtraggio deve essere condotta utilizzando un anchor point centrale quindi il pixel che subisce l'operazione di filtraggio è quello che si trova al centro della finestra.

L'immagine viene gestita attraverso il Buffer Line che restituisce i pixel di 3 righe allineati, pronti per essere inviati al moltiplicatore.

Viene rivolta particolare attenzione per i pixel che si trovano lungo i bordi e che nella finestra di convoluzione subiscono zero padding, che viene garantito dall'inizializzazione a 0 del buffer e dei vettori del componente.

Il filtraggio vero e proprio avviene attraverso l'operazione di convoluzione: i pixel dell'immagine vengono moltiplicati con il coefficiente corrispondente del filtro e infine tutti i prodotti parziali vengono sommati insieme. Il risultato sarà il pixel centrale filtrato con il Filtro Gaussiano o Gaussian Blur che ha come coefficienti:

1	2	1
2	4	2
1	2	1

Per il testing abbiamo optato per diverse immagini, tra cui alcune figure semplici che fanno risaltare molto l'influenza del filtro.

Per processare le immagini sono stati scritti diversi script MATLAB tra cui

- uno per creare semplici immagini da impiegare nel circuito;
- uno per ridimensionare immagini (prese da internet) nel formato 32x32 pixel;
- uno per estrarre le componenti numeriche di ogni immagine che saranno poi pronte per essere convertite in bit e inserite come input nel circuito;
- uno per rielaborare le componenti numeriche e ricostruire l'immagine filtrata.

Capitolo 2

Componenti Base

In questo capitolo vengono descritti i blocchi aritmetici fondamentali progettati ed implementati per costituire la base delle elaborazioni successive.

2.1 Full Adder

L'elemento atomico per le operazioni aritmetiche è il Full Adder, un circuito combinatorio in grado di calcolare la somma di tre bit: due addendi (A , B) e un riporto in ingresso (C_{in}).

2.1.1 Logica Implementativa

Per l'implementazione su FPGA è stata scelta una descrizione *behavioral*. Sebbene le equazioni canoniche siano:

$$S = A \oplus B \oplus C_{in} \quad (2.1)$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B)) \quad (2.2)$$

Il codice VHDL sfrutta il segnale di propagazione P per ottimizzare l'inferenza della logica di riporto (Carry Logic), fondamentale per le prestazioni temporali:

- Se $P = 1$ ($A \neq B$), il riporto C_{out} è determinato da C_{in} .
- Se $P = 0$ ($A = B$), il riporto C_{out} è uguale ad A .

```

1  entity full_adder is
2      port (
3          a, b, cin : in  STD_LOGIC;
4          s, cout   : out STD_LOGIC
5      );
6  end entity full_adder;
7
8  architecture behavioral of full_adder is
9      signal p : STD_LOGIC;
10     begin
11         -- Calcolo Propagate
12         p <= a xor b;
13         s <= p xor cin;
14
15         -- Mux-based Carry logic (Efficiente su FPGA)
16         cout <= cin when p = '1' else a;
17     end architecture behavioral;

```

Listing 2.1: Entity e Architecture del Full Adder

2.2 N-Bit Ripple Carry Adder (RCA)

Per sommare vettori di bit, la struttura più semplice è il Ripple Carry Adder, ottenuto collegando in cascata N istanze di Full Adder.

2.2.1 Architettura Parametrica

Il componente è stato reso generico attraverso il parametro 'N'. L'uso del costrutto `generate` permette di istanziare dinamicamente la catena di addizionatori in base alla larghezza di bit desiderata a tempo di sintesi.

- **Vantaggi:** Semplicità implementativa e occupazione d'area ridotta.
- **Svantaggi:** Ritardo di propagazione lineare ($T_{delay} \propto N$), poiché il riporto deve attraversare l'intera catena dal LSB al MSB.

```

1  entity ripple_carry_adder is
2      generic ( N : POSITIVE );
3      port (
4          a, b : in  std_logic_vector(N-1 downto 0);
5          cin  : in  std_logic;

```

```

6      s      : out std_logic_vector(N-1 downto 0);
7      cout   : out std_logic
8    );
9  end entity ripple_carry_adder;
10
11  architecture structural of ripple_carry_adder is
12    signal c : std_logic_vector(N downto 0);
13    begin
14      c(0) <= cin; -- Carry in iniziale
15
16      loop_n: for i in 0 to N - 1 generate
17        fa_i: entity work.full_adder
18        port map (
19          a    => a(i),
20          b    => b(i),
21          cin  => c(i),
22          s    => s(i),
23          cout => c(i + 1) -- Chain connection
24        );
25      end generate loop_n;
26
27      cout <= c(N); -- Ultimo riporto
28  end architecture structural;

```

Listing 2.2: Ripple Carry Adder parametrico

2.3 Strategie di Testing

La verifica dei componenti aritmetici ha seguito due approcci distinti in base alla complessità dello spazio degli stati.

2.3.1 Unit Testing: Full Adder (Exhaustive)

Poiché il Full Adder ha solo 3 ingressi ($2^3 = 8$ casi totali), è stato possibile implementare un test esaustivo iterando su una **Truth Table** completa. La testbench è *self-checking*: confronta l'uscita del DUT (Device Under Test) con il valore atteso memorizzato nella tabella.

```

1  type truth_table_type is array (0 to 7) of std_logic_vector
2    (4 downto 0);
3
4  constant TRUTH_TABLE : truth_table_type := (
5    "000" & "00", -- 0+0+0 = 0 (C=0)

```



```

5  "011" & "10", -- 0+1+1 = 0 (C=1)
6  "111" & "11", -- 1+1+1 = 1 (C=1)
7  -- ... altri casi ...
8  others => (others => '0')
9  );
10
11 -- Loop di simulazione
12 -- (Codice omissso per brevita', itera su TRUTH_TABLE)

```

Listing 2.3: Testbench Full Adder con Tabella della Verità

2.3.2 Unit Testing: Ripple Carry Adder (Directed)

Per l'addizionatore a N bit, verificare tutte le combinazioni (2^{2N+1}) è impraticabile. Si è optato per un *Directed Testing* focalizzato sui "Corner Cases" (casi limite) critici per la propagazione del riporto.

```

1  constant TRUTH_TABLE : truth_table_type := (
2  -- Caso Base: 5 + 5 = 10
3  2 => std_logic_vector(x"05") & std_logic_vector(x"05") &
   '0' & ...
4
5  -- Caso Overflow (Carry Out generation):
6  -- 255 (FF) + 1 (01) = 0 (00) con Carry=1
7  3 => std_logic_vector(x"FF") &
   std_logic_vector(x"01") &
8  '0' & -- Cin
9  '1' & -- Expected Cout
10 std_logic_vector(x"00"), -- Expected Sum
11
12 -- Altri casi...
13 others => (others => '0')
14 );
15

```

Listing 2.4: Corner Cases per RCA

Gli scenari validati includono:

1. **Overflow:** Verifica che il riporto esca correttamente dal MSB ($C_{out} = 1$).
2. **Propagazione Completa:** Verifica il caso peggiore di ritardo, dove un riporto generato al bit 0 deve propagarsi fino all'uscita C_{out} (es. $011..1 + 000..1$).

Capitolo 3

Il moltiplicatore

3.1 Implementazione

La differenza principale con il moltiplicatore di Booth è che non si va a suddividere in gruppi di bit il moltiplicatore, ma si vanno a codificare tutti i possibili valori che possono assumere i coefficienti del filtro.

La nostra implementazione va a sfruttare i $2^4 = 16$ casi possibili per ogni coefficiente del filtro $F_{x,y}$.

La buffer line scorre sull'immagine e invia i dati al moltiplicatore, nove alla volta secondo una finestra 3x3, i coefficienti del filtro invece restano fissi e vengono codificati.

Di seguito la entity:

```
1  entity booth_multiplier is
2  generic(
3    componente_immagine : POSITIVE := 8;
4    coefficiente_filtro : POSITIVE := 4;
5    somma : POSITIVE := 12
6  );
7  port (
8    P_1_1, P_1_2, P_1_3 : in std_logic_vector(
9      componente_immagine-1 downto 0);
10   P_2_1, P_2_2, P_2_3 : in std_logic_vector(
11     componente_immagine-1 downto 0);
12   P_3_1, P_3_2, P_3_3 : in std_logic_vector(
13     componente_immagine-1 downto 0);
14
15   F_1_1, F_1_2, F_1_3 : in std_logic_vector(
16     coefficiente_filtro-1 downto 0);
```

```

17 F_2_1, F_2_2, F_2_3 : in std_logic_vector(
18   coefficiente_filtro-1 downto 0);
19 F_3_1, F_3_2, F_3_3 : in std_logic_vector(
20   coefficiente_filtro-1 downto 0);
21
22 M_1_1, M_1_2, M_1_3 : out std_logic_vector(
23   somma-1 downto 0);
24 M_2_1, M_2_2, M_2_3 : out std_logic_vector(
25   somma-1 downto 0);
26 M_3_1, M_3_2, M_3_3 : out std_logic_vector(
27   somma-1 downto 0)
28 );
29 end entity booth_multiplier;
30

```

Listing 3.1: Entity del moltiplicatore

Le entrate si suddividono in questo modo:

- i segnali $P_{x,y}$ sono i pixel dell'immagine che hanno dimensione 8-bit
- i segnali $F_{x,y}$ sono i coefficienti del filtro con dimensione 4-bit

Se il moltiplicando e il moltiplicatore sono composti rispettivamente da m ed n bit il risultato del prodotto andrà espresso in $m + n$ -bit.

Le uscite sono invece i segnali $M_{x,y}$ con dimensione $m + n = 8 + 4 = 12$ -bit.

Il moltiplicatore segue questo funzionamento:

- L'operazione principale che abbiamo effettuato è la codifica dei sedici possibili valori che può assumere ogni coefficiente del filtro;
- Si codificano i due prodotti parziali $P \cdot P_A_{x,y}$ e $P \cdot P_B_{x,y}$ per ogni pixel e relativo coefficiente: dato l'elevato numero dei casi possibili diventa complesso offrire una codifica diretta quindi si utilizzano A e B che vengono ciascuno moltiplicati per $P_{x,y}$ separatamente e si ottengono i segnali $P \cdot P_A_{x,y}$ e $P \cdot P_B_{x,y}$ secondo la seguente tabella:

Case filtro	Numero da codificare	A	B	P_P_A_x_y	P_P_B_x_y
0000	0	0	0	000000000000	000000000000
0001	1	1	0	0000 P_x_y	000000000000
0010	2	-2	4	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	00 P_x_y 00
0011	3	-1	4	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	00 P_x_y 00
0100	4	0	4	000000000000	00 P_x_y 00
0101	5	1	4	0000 P_x_y	00 P_x_y 00
0110	6	-2	8	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	0 P_x_y 000
0111	7	-1	8	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	0 P_x_y 000
1000	-8	0	-8	000000000000	N_P_x_y 000
1001	-7	1	-8	0000 P_x_y	N_P_x_y 000
1010	-6	-2	-4	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	N_P_x_y(8) N_P_x_y 00
1011	-5	-1	-4	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	N_P_x_y(8) N_P_x_y 00
1100	-4	0	-4	000000000000	N_P_x_y(8) N_P_x_y 00
1101	-3	1	-4	0000 P_x_y	N_P_x_y(8) N_P_x_y 00
1110	-2	-2	0	N_P_x_y(8) N_P_x_y(8) N_P_x_y 0	000000000000
1111	-1	-1	0	N_P_x_y(8) N_P_x_y(8) N_P_x_y(8) N_P_x_y	000000000000
others	0	0	0	000000000000	000000000000

Da notare che $N_P_x_y$ ha dimensione 9-bit ed esprime il complemento a due del pixel P_x_y , quindi corrisponde alla sua inversione dei singoli bit e alla somma di '1' attraverso un Ripple Carry Adder (in pratica rappresenta la versione negativa del segnale iniziale), e si utilizza quando i prodotti parziali $P_P_A_x_y$ e $P_P_B_x_y$ sono negativi.

Da ricordare che $P_P_A_x_y$ e $P_P_B_x_y$ devono essere di 12-bit quindi vanno gestiti efficientemente i prodotti da effettuare attraverso gli shift sinistri e l'estensione che deve portare ogni segnale alla dimensione giusta.

Questa operazione all'interno del codice risulta in nove case statement (uno per ogni coefficiente del filtro) in cui si codificano le componenti $P_P_A_x_y$ e $P_P_B_x_y$ per tutti i coefficienti del filtro.

- L'ultimo passaggio è la somma fra le componenti $P_P_A_x_y$ e $P_P_B_x_y$ calcolata per tutti gli ingressi attraverso dei semplici circuiti sommatore Ripple Carry.

Il risultato delle somme saranno i nove operandi a 12-bit pronti per essere inviati al circuito Carry Save Adder Tree che provvederà a sommarli insieme e a determinare il valore del pixel filtrato, corrispondente al pixel P_2_2 in ingresso.

3.1.1 Alcune note sui prodotti

I prodotti vengono effettuati in modo intelligente, provando a sfruttare operazioni immediate come shift sinistri e negazioni.

Fra i prodotti da calcolare si privilegiano quelli per 0, +1, -1, +2, -2, +4, -4 e così via, nello medesimo modo in cui si calcolano nel moltiplicatore di Booth.

Questi prodotti sono semplici da ottenere:

- Prodotto per 0 => Il prodotto parziale avrà tutti i bit a 0;
 - Prodotto per 1 => Il prodotto parziale sarà uguale al moltiplicando;
 - Prodotto per -1 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit e a cui viene sommato 1;
 - Prodotto per 2 => Il prodotto parziale sarà uguale al moltiplicando che subisce uno shift sinistro;
 - Prodotto per -2 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e uno shift sinistro;
 - Prodotto per 4 => Il prodotto parziale sarà uguale al moltiplicando che subisce due shift sinistri;
 - Prodotto per -4 => Il prodotto parziale sarà uguale al moltiplicando che subisce l'inversione dei bit, la somma di 1 e due shift sinistri;
- e così via.

3.1.2 Un esempio

La strategia è quella di utilizzare A e B per ottenere i prodotti combinando i parziali P_P_A_x_y e P_P_B_x_y attraverso la loro somma finale.

Ad esempio nel caso in cui il coefficiente abbia valore 5 (quindi 0101) si ricorre a codificare il prodotto per A=1 e per B=4 e si determinano i parziali da sommare insieme:

- $P_P_A_x_y = P_x_y * 1 \Rightarrow P_P_A_x_y = P_x_y$
- $P_P_B_x_y = P_x_y * 4 \Rightarrow$ Si effettuano due shift sinistri a $P_x_y \Rightarrow P_P_B_x_y = P_x_y \ 0 \ 0$

Allo stesso tempo si estendono i parziali per portarli alla dimensione 12-bit ricordando che P_x_y ha dimensione 8-bit.

I parziali per il caso in cui il coefficiente del filtro corrisponde a 1010 saranno:

- $P_P_A_x_y = 0 \ 0 \ 0 \ 0 \ P_x_y$
- $P_P_B_x_y = 0 \ 0 \ P_x_y \ 0 \ 0$

Che verranno sommati insieme con un Ripple Carry Adder.

Lo stesso esempio corrisponde ad avere

$$P_x_y * 5 \Rightarrow (P_x_y * 1) + (P_x_y * 4)$$

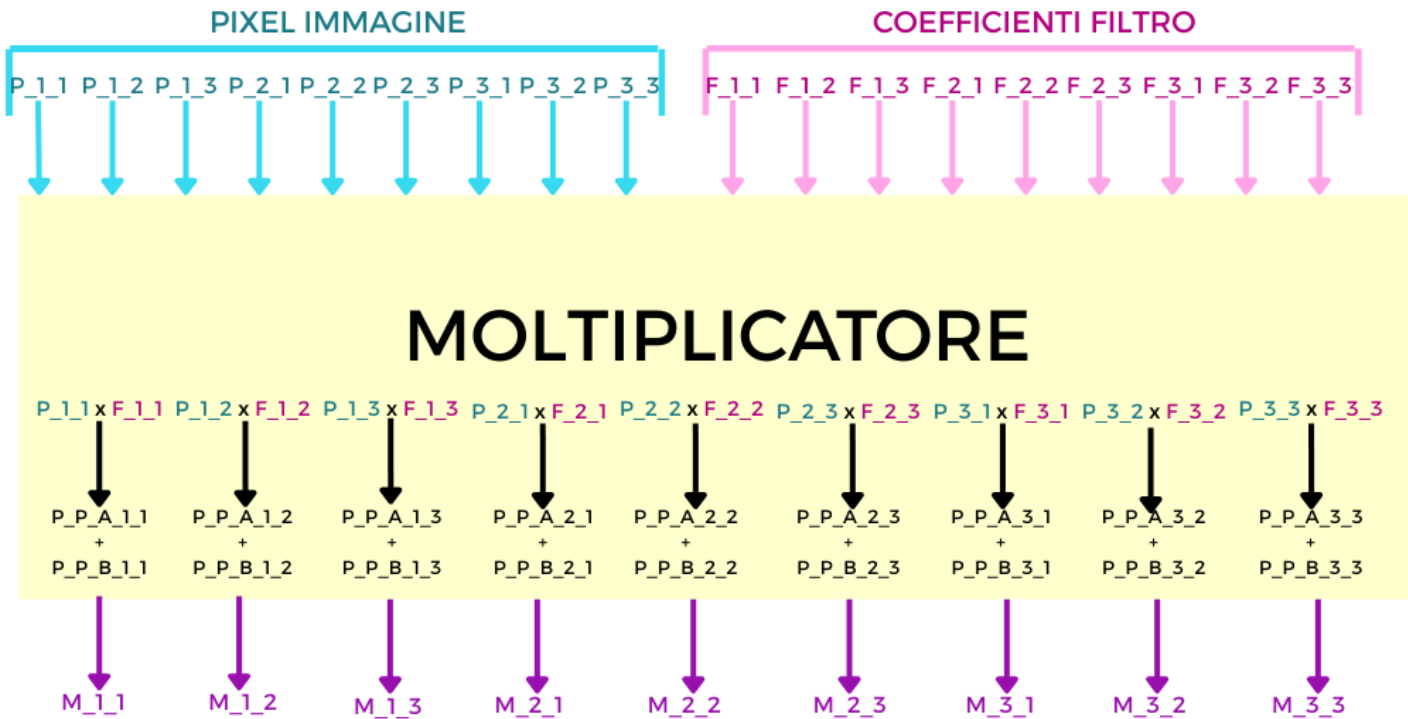
$$P_x_y * 5 \Rightarrow P_x_y + (P_x_y * 4)$$

$$P_x_y * 5 \Rightarrow P_x_y + P_x_y + P_x_y + P_x_y + P_x_y$$

Per ognuna delle nove coppie pixel immagine-coefficiente filtro si determinano le somme dei parziali e si inviano come input per il Carry Save Adder Tree che provvederà a fare la somma fra tutti gli operandi.

3.2 Il circuito moltiplicativo

Il circuito finale avrà questa struttura:



3.3 Il testing

Fra alcuni casi testati figurano:

- Il massimo caso positivo, in cui i pixel valgono 255 e i coefficienti valgono 7

```

1      --test sui massimi valori possibili
2      -- immagine=255, filtro = +7
3      --risultati attesi = 1785
4      p <= std_logic_vector(to_unsigned(255, comp_i
5      ));
6      f <= std_logic_vector(to_signed(7, coeff_f));
7      wait for 50 ns;

```

- Il massimo caso negativo in cui i pixel valgono 255 e i coefficienti valgono -8

```

1      --immagine = 255, filtro = -8
2      --risultato atteso = -2040
3      p <= std_logic_vector(to_unsigned(255, comp_i));
4      f <= std_logic_vector(to_signed(-8, coeff_f))
5      ;
6      wait for 50 ns;

```

- Caso in cui i pixel restano fissi a 255 e i coefficienti del filtro variano per tutti i valori possibili: da -8 a 7

```

1      p <= std_logic_vector(to_unsigned(255, 8));
2      for i in -8 to 7 loop
3          f <= std_logic_vector(to_signed(i,
4      coeff_f));
5      wait for 50 ns;
6      end loop;
7      wait for 20 ns;

```


Si allega ora la simulazione del testbench effettuato in cui figurano i risultati correttamente calcolati:

> ♡ P_1_1[7:0]	255	0	255	0	255							
> ♡ P_1_2[7:0]	255	0	255	0	255							
> ♡ P_1_3[7:0]	255	0	255	0	255							
> ♡ P_2_1[7:0]	255	0	255	0	255							
> ♡ P_2_2[7:0]	255	0	255	0	255							
> ♡ P_2_3[7:0]	255	0	255	0	255							
> ♡ P_3_1[7:0]	255	0	255	0	255							
> ♡ P_3_2[7:0]	255	0	255	0	255							
> ♡ P_3_3[7:0]	255	0	255	0	255							
> ♡ F_1_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_1_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_1_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_2_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_2_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_2_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_3_1[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_3_2[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ F_3_3[3:0]	-2	0	7	-8	-7	-6	-5	-4	-3	-2		
> ♡ M_1_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_1_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_1_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_2_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_2_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_2_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_3_1[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_3_2[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510
> ♡ M_3_3[11:0]	-510	0	1785	-2040	0	-2040	-1785	-1530	-1275	-1020	-765	-510

Capitolo 4

Il Carry Save Adder Tree

4.1 Implementazione

Dopo aver moltiplicato i pixel uscenti dalla finestra di registri per i corrispondenti valori del filtro si otterranno dunque nove operandi da 12-bit ciascuno.

Per ottenere il pixel filtrato sarà ora necessario sommare tutti e nove gli operandi per poi avere in uscita il risultato atteso.

Per poter fare ciò si hanno diverse strade, tra cui un classico sommatore “carta e penna” o una somma a due a due. Questi metodi, per quanto semplici da implementare, hanno però il grosso svantaggio di essere molto lenti ad effettuare il calcolo. Esistono invece metodi più efficienti che sfruttano più risorse HW ma che portano a terminare l’operazione in un tempo molto più breve. Uno tra questi è l’approccio detto “Carry Save”. Questa metodologia consiste nel sommare gli operandi a due a due o a tre a tre, creando in uscita due diversi vettori contenenti rispettivamente i bit delle somme e i bit dei riporti.

Questo approccio interrompe dunque la catena di trasporto del riporto per ogni somma, andando ad iterare questa procedura fino ad ottenere in uscita due soli vettori da sommare. Sarà solo a questo punto che si avrà una sola catena di riporto, andando dunque a risparmiare tanto tempo a livello complessivo. Essendo questo progetto incentrato sull’efficienza, dando il giusto peso all’utilizzo delle risorse, è stato dunque scelto un approccio Carry Save per l’albero di somma mentre, per lo step finale, è stato scelto una somma mediante un modulo Carry Select Adder. Questo per via del

numero di bit favorevole che ha permesso di adottare il seguente approccio senza avere sbilanciamenti o verso le troppe risorse o verso un tempo di elaborazione eccessivamente alto.

L'albero di somma sarà formato da nove operandi in ingresso corrispondenti ai prodotti parziali e saranno rappresentati da altrettanti segnali in `std_logic_vector` a 12-bit. In uscita, per come verrà spiegato in seguito, si avrà un segnale in `std_logic_vector` a 16-bit.



Mentre, in VHDL, il blocco sarà descritto nel seguente modo:

```

1  entity carry_save_adder_tree is
2      generic (N : POSITIVE := 12);
3
4      port (
5          i_1_1, i_1_2, i_1_3 : in  std_logic_vector(
6              N-1 downto 0);
7          i_2_1, i_2_2, i_2_3 : in  std_logic_vector(

```

```

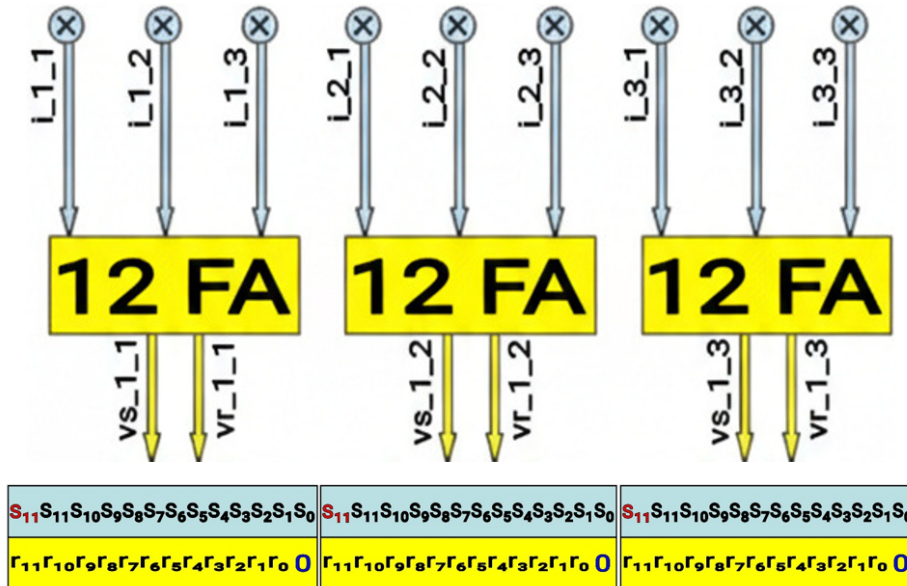
8         N-1 downto 0);
9         i_3_1, i_3_2, i_3_3 : in  std_logic_vector(
10            N-1 downto 0);
11         sum                    : out std_logic_vector(
12            N+3 downto 0)
13     );
14 end carry_save_adder_tree;

```

Listing 4.1: Entity del circuito sommatore

Analizzando invece il comportamento interno del sommatore, in un primo livello i segnali verranno raggruppati a tre a tre e sommati bit a bit da un ulteriore blocco composto da dodici Full Adder separati.

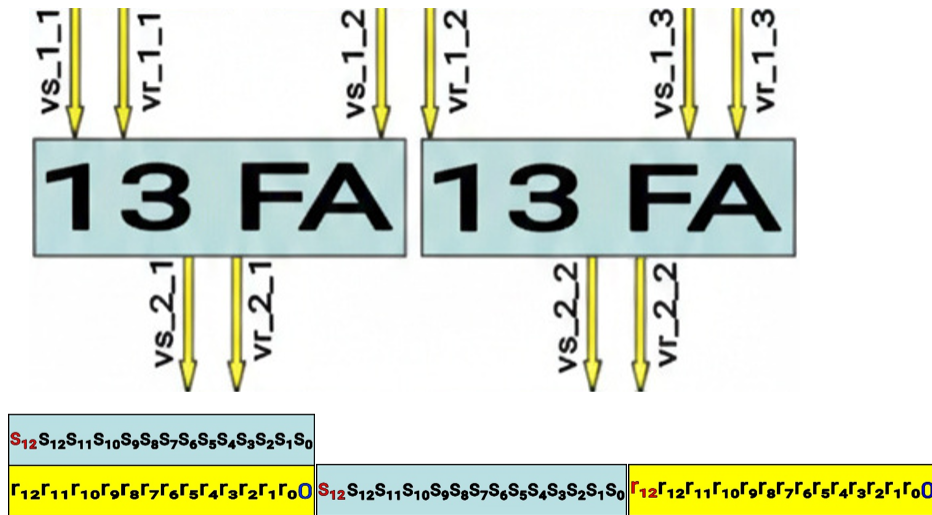
I bit in posizione i -esima daranno in uscita un bit di somma e uno di riporto che andranno nei due rispettivi vettori descritti in precedenza, senza avere dunque una catena di riporto. Dopo aver ottenuto i 12-bit di somma e riporto, il vettore dei riporti verrà shiftato a sinistra di una posizione estendendo con lo zero a destra (il LSB) e per avere lo stesso numero di bit il vettore somma verrà esteso con segno a sinistra del MSB. Questa procedura verrà ripetuta tre volte ottenendo complessivamente tre vettori di somme e tre vettori di riporti ognuno dei quali sarà a 13-bit. Graficamente la condizione può essere così descritta:



È importante sottolineare che, l'estensione con lo zero a destra e l'estensione con segno a sinistra non richiede risorse aggiuntive ma solo un collegamento rispettivamente verso GND e verso il MSB.

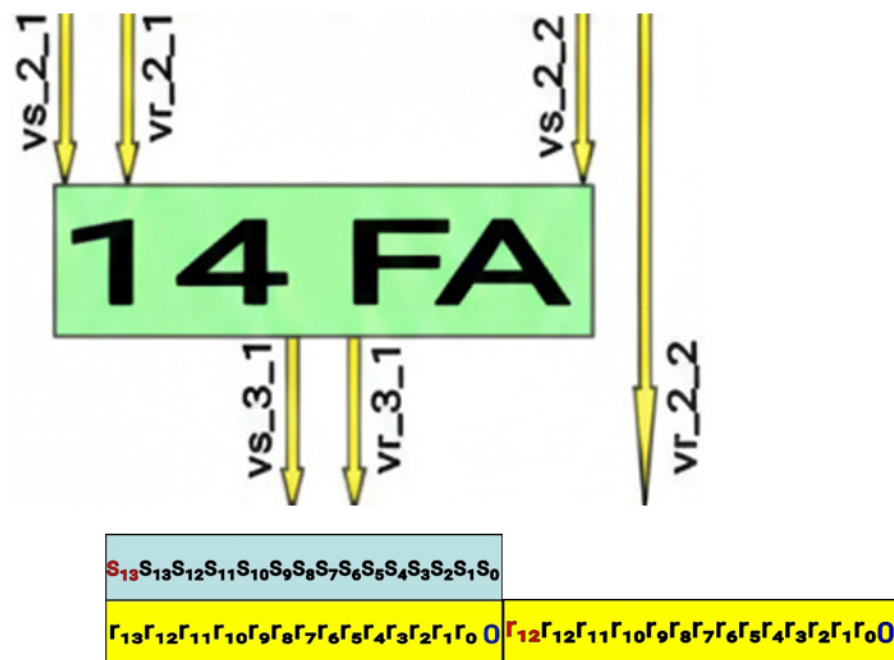
Ottenuti i sei nuovi vettori, verrà replicata la procedura andando a raggruppare i segnali a tre a tre. Nello specifico, per il primo raggruppamento verranno presi i due segnali uscenti dal primo blocchetto e il primo del secondo blocchetto. L'altro raggruppamento sarà invece formato dal secondo segnale uscente dal secondo blocchetto e da entrambi i segnali uscenti dal terzo blocchetto.

Presi i due raggruppamenti di segnali, essi andranno in ingresso ai due nuovi blocchi sommatori. In questo secondo livello i Full Adder non saranno più dodici come prima ma tredici per via dello shift e della conseguente estensione con segno. A somme effettuate, in uscita si avranno quattro nuovi segnali, due riferiti ai vettori somma e due ai vettori riporto. Tre dei quattro segnali saranno a 14-bit e saranno i segnali di ingresso del successivo livello di somma. Il quarto segnale, quello relativo al vettore dei riporti del secondo blocchetto, avrà una doppia estensione con segno a sinistra del MSB. Esso sarà infatti l'ingresso dell'ultimo livello di somma e avrà un'estensione totale pari a 15-bit. Graficamente la situazione sarà la seguente:

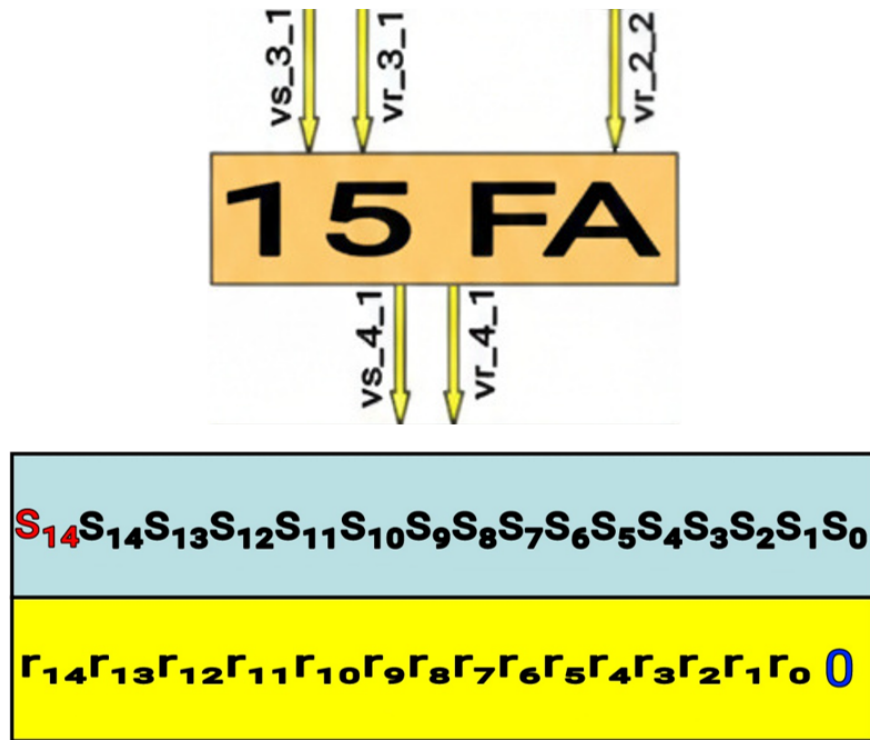


Passando al successivo livello di somma, verranno presi i tre vettori da 14-bit e inviati in ingresso al blocchetto di somma composto da quattordici Full Adder mentre il segnale da 15-bit proseguirà verso il livello finale. Dal

blocchetto di quattordici Full Adder usciranno ulteriori due segnali a 15-bit, uno relativo ad un vettore somma e uno al vettore riporti con le canoniche estensioni con lo zero e con segno nelle opportune posizioni. A livello grafico:



A questo si arriverà ad avere tre vettori da 15-bit ciascuno, rispettivamente uno di somma e due di riporti. I tre vettori andranno in ingresso al livello di somma finale composto da quindi Full Adder che darà come uscita i due vettori finali di somma e riporto da 16-bit. A livello grafico:

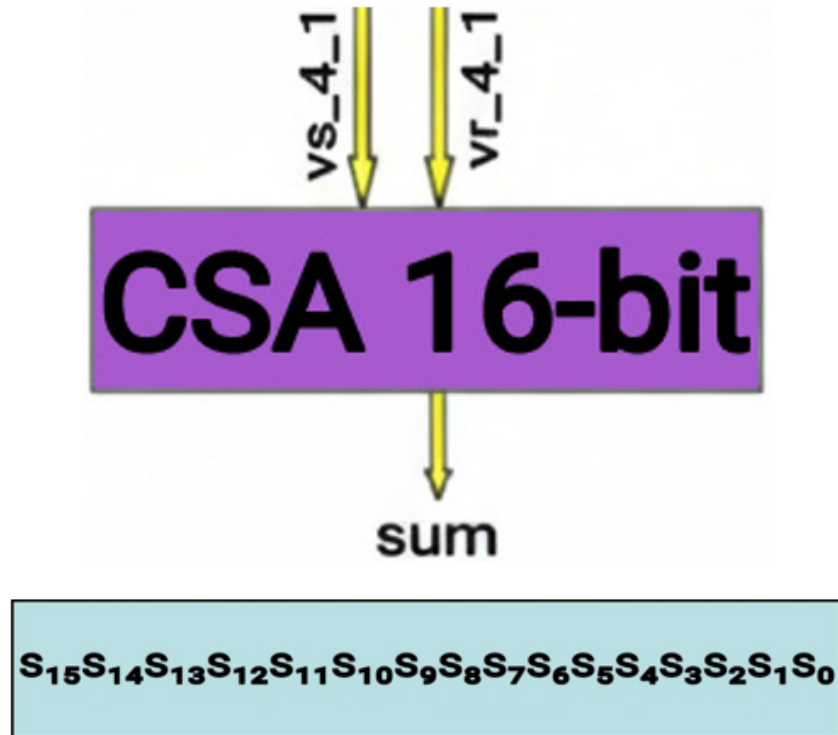


Infine si avranno due vettori finali da 16-bit che, una volta sommati, daranno luogo al risultato finale. Prima di ottenere ciò andranno sommati bit a bit i due vettori con un metodo classico, e quindi con una catena di riporto.

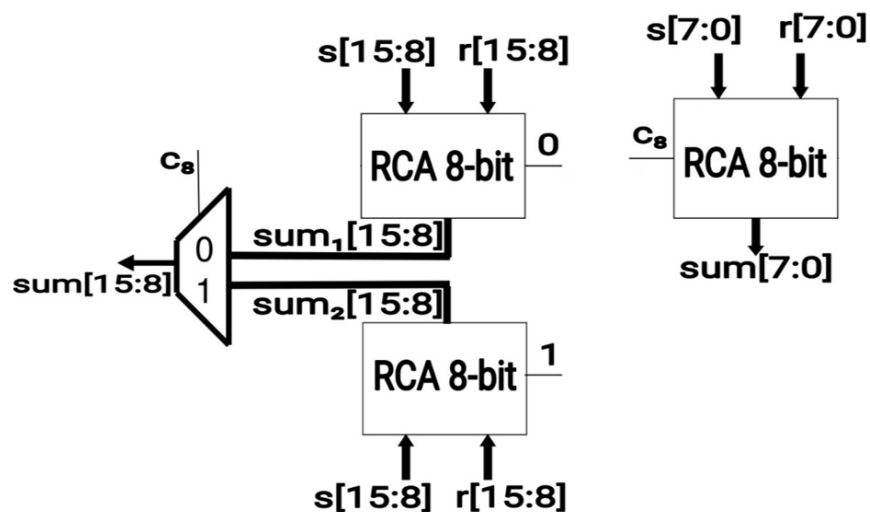
4.2 Il Carry Select Adder

Per la seguente somma è stato scelto un Carry Select Adder con moduli di Ripple Carry Adder ad 8-bit ciascuno, trovando dunque il giusto compromesso tra risorse, potenza dissipata e tempo di elaborazione. Il Carry Select Adder, inoltre, sarà una versione semplificata rispetto a quella canonica poiché non faremo uso dell'ultimo riporto in uscita. Questo perché il risultato sarà a 16-bit totali e già esteso nel modo opportuno. Dunque, il riporto non verrà sfruttato in quanto i valori massimo e minimo ottenibili rientreranno senza problemi all'interno del range permesso dai 16-bit con

rappresentazione signed. Essendo inoltre il riporto un segnale di selezione per il MUX del riporto, neanche quest'ultimo verrà integrato nel circuito.

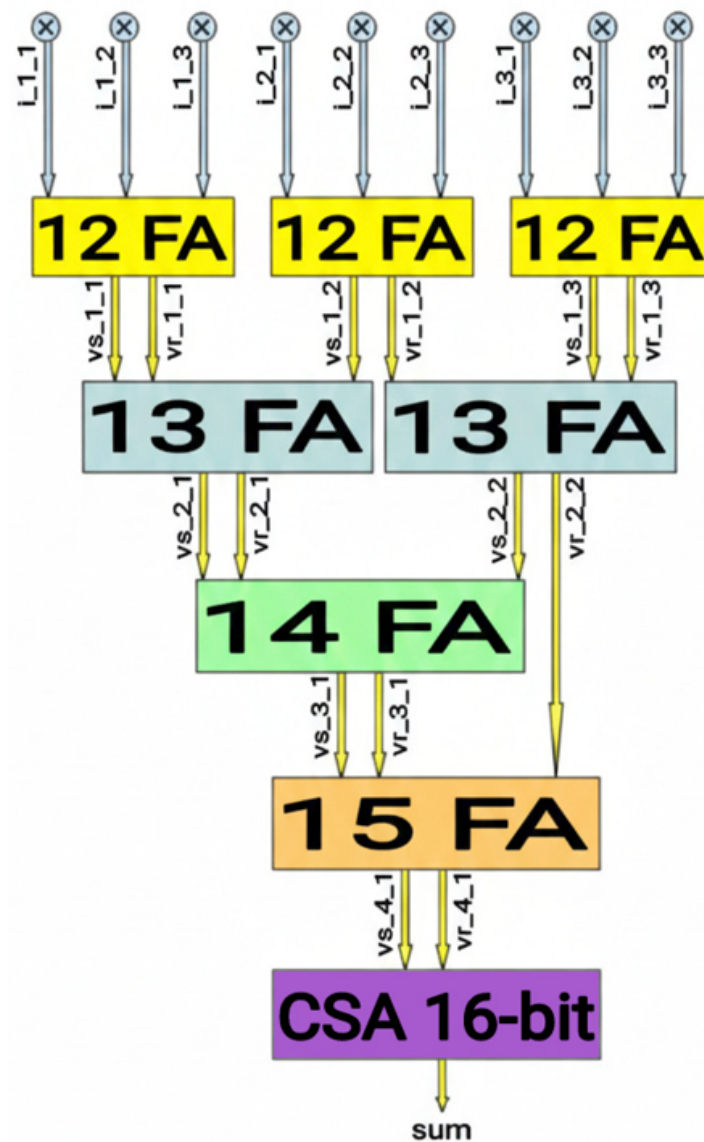


L'implementazione del CSA in versione semplificata sarà la seguente:



4.3 Il circuito finale

Unendo tutte le varie parti si otterrà dunque il seguente circuito, corrispondente ad un sommatore a nove operandi da dodici bit ciascuno di tipo signed con approccio Carry Save Adder e sommatore a Carry Select Adder finale:



4.4 Il testing

Verrà fatto adesso un rapido testbench sul modulo appena descritto, andando a confermare i risultati ottenuti nel worst case positivo e negativo, assicurandosi di non avere problemi in uscita. Immaginando di avere il valore massimo

sui pixel, ovvero +255 a 8-bit in rappresentazione unsigned pari “11111111”, verrà analizzato il caso in cui i coefficienti del filtro saranno tutti pari al massimo valore positivo e il caso in cui saranno tutti pari al minimo valore negativo.

4.4.1 Caso positivo

Essendo i coefficienti del filtro a 4-bit rappresentati in complemento a due, essi potranno assumere come valore massimo il numero decimale +7. Effettuando quindi un filtraggio tra nove pixel pari a +255 a 8-bit unsigned e nove coefficienti pari a +7 a 4-bit signed, in uscita otterremo nove operandi tutti pari +1785 a 12-bit di tipo signed. Andando poi ad effettuare la somma, in uscita si dovrà ottenere il valore +16065 a 16-bit di tipo signed. In forma tabellare:

+255	+255	+255
+255	+255	+255
+255	+255	+255

+7	+7	+7
+7	+7	+7
+7	+7	+7

+1785	+1785	+1785
+1785	+1785	+1785
+1785	+1785	+1785

16065

Visualizzando il timing su Vivado:

Name	Value	0 ps	1 ps	2 ps	3 ps	4 ps	5 ps	6 ps
> tb_i_1_1[11:0]	1785							1785
> tb_i_1_2[11:0]	1785							1785
> tb_i_1_3[11:0]	1785							1785
> tb_i_2_1[11:0]	1785							1785
> tb_i_2_2[11:0]	1785							1785
> tb_i_2_3[11:0]	1785							1785
> tb_i_3_1[11:0]	1785							1785
> tb_i_3_2[11:0]	1785							1785
> tb_i_3_3[11:0]	1785							1785
> tb_sum[15:0]	16065							16065

4.4.2 Caso negativo

Essendo i coefficienti del filtro a 4-bit rappresentati in complemento a due, essi potranno assumere come valore minimo il numero decimale -8. Effettuando quindi un filtraggio tra nove pixel pari a +255 a 8-bit unsigned e nove coefficienti pari a -8 a 4-bit signed, in uscita otterremo nove operandi tutti pari a -2040 a 12-bit di tipo signed. Andando poi ad effettuare la somma, in uscita si dovrà ottenere il valore -18360 a 16-bit di tipo signed. In forma tabellare:

+255	+255	+255
+255	+255	+255
+255	+255	+255

-8	-8	-8
-8	-8	-8
-8	-8	-8

-2040	-2040	-2040
-2040	-2040	-2040
-2040	-2040	-2040

-18360

Visualizzando il timing su Vivado:

Name	Value	183,030 ps	183,031 ps	183,032 ps	183,033 ps	183,034 ps	183,035 ps	183,036 ps
>  tb_i_1_1[11:0]	-2040							-2040
>  tb_i_1_2[11:0]	-2040							-2040
>  tb_i_1_3[11:0]	-2040							-2040
>  tb_i_2_1[11:0]	-2040							-2040
>  tb_i_2_2[11:0]	-2040							-2040
>  tb_i_2_3[11:0]	-2040							-2040
>  tb_i_3_1[11:0]	-2040							-2040
>  tb_i_3_2[11:0]	-2040							-2040
>  tb_i_3_3[11:0]	-2040							-2040
>  tb_sum[15:0]	-18360							-18360

Capitolo 5

BufferLine

5.1 Buffer e Pipeline: L'architettura Sliding Window

La trasformazione di un'immagine in un contesto spaziale bidimensionale è il compito principale del modulo `buffer_line`. Questo blocco è responsabile della creazione della "finestra scorrevole" (Sliding Window) 3×3 necessaria per l'operazione di convoluzione.

5.1.1 Architettura del Dataflow

Il modulo implementa una struttura a registri a scorrimento (Shift Registers) interconnessi con buffer di linea (FIFO). L'obiettivo è rendere disponibili contemporaneamente, in un singolo ciclo di clock, i 9 pixel adiacenti necessari al filtro:

""Immagine da inserire""

$$\text{Window} = \begin{bmatrix} d_{00} & d_{01} & d_{02} \\ d_{10} & d_{11} & d_{12} \\ d_{20} & d_{21} & d_{22} \end{bmatrix}$$

I dati in ingresso dall'interfaccia AXI-Stream vengono inseriti nel registro d_{00} . Ad ogni colpo di clock, se l'abilitazione della pipeline è attiva, i dati scorrono verso destra. Al termine della prima riga della finestra (d_{02}), il dato entra in un *Line Buffer* che lo ritarda di un numero di cicli pari alla larghezza rimanente dell'immagine, per poi riapparire all'inizio della seconda riga (d_{10}).

Questa architettura garantisce che, una volta riempita la pipeline (latenza iniziale), i registri contengano sempre pixel spazialmente adiacenti, nonostante arrivino temporalmente sequenziali.

5.1.2 Implementazione degli Shift Register Arrays

Le FIFO necessarie per memorizzare le righe dell'immagine sono realizzate utilizzando array parametrici di vettori logici. La dimensione di questi buffer è critica e dipende dalla larghezza dell'immagine (N_{col}) e dalla dimensione del kernel ($K_{dim} = 3$).

La profondità del buffer è calcolata come:

$$\text{Buffer Depth} = N_{col} - K_{dim}$$

Nel codice VHDL (Listing 5.1), viene definito un tipo array per istanziare i due buffer di linea necessari per una finestra 3×3 .

```

1  -- Definizione dei tipi per il buffer
2  -- Esempio: Image Width (32) - Window Size (3) = 29
   elementi di buffer
3  -- La dimensione e' parametrica basata su ncol
4  type reg_array is array (ncol-4 downto 0) of
   std_logic_vector(7 downto 0);
5  signal buffer1, buffer2 : reg_array;
6
7  -- Processo di scorrimento (Line Buffer 1)
8  process(clk)
9  begin
10     if rising_edge(clk) then
11         if pipeline_en = '1' then
12             -- Ingresso dal registro finale della riga precedente
               (d02)
13             buffer1(0) <= d02_s;
14             -- Shift dell'intera riga nel buffer
15             for j in 1 to ncol-4 loop
16                 buffer1(j) <= buffer1(j-1);
17             end loop;
18
19             -- L'uscita del buffer alimenta la riga successiva (d10
               )
20             d10_s <= buffer1(ncol-4);
21         end if;
22     end if;

```

```
23 end process;
```

Listing 5.1: Definizione e processo di shift dei Line Buffer

5.1.3 Gestione del Flush e Padding

Una criticità nei filtri digitali è la gestione dei bordi dell'immagine. Quando l'ultimo pixel valido entra nella pipeline, la finestra di elaborazione non ha ancora terminato il suo lavoro: l'ultimo pixel deve infatti scorrere fino a raggiungere la posizione centrale (d_{11}) affinché l'operazione di filtraggio sia completa per tutto il frame.

Per gestire questa fase, il sistema implementa una logica di *Zero Padding* attivata dallo stato di **FLUSH** della FSM.

Logica di Inserimento Zeri

Quando il segnale `flush` è attivo, l'ingresso fisico della pipeline viene disaccoppiato dall'interfaccia AXI e forzato a zero. Questo permette di "spingere" i dati validi residui attraverso i registri e i buffer di linea senza introdurre nuovi valori che corromperebbero il calcolo.

```
1  if rising_edge(clk) and pipeline_en = '1' then
2  -- Se siamo in fase di FLUSH, carichiamo zeri (Padding)
3  -- per svuotare la pipeline dai pixel validi residui
4  if flush_state = '1' then
5      d00_s <= (others => '0');
6  else
7      -- Durante il normale funzionamento, acquisiamo il dato
      AXI
8      d00_s <= s_axis_tdata;
9  end if;
10
11 -- Shift dei registri di finestra (Row 0)
12 d01_s <= d00_s;
13 d02_s <= d01_s;
14 end if;
```

Listing 5.2: Multiplexer di ingresso per gestione Flush e Padding

Questa tecnica assicura che l'immagine in uscita abbia esattamente le stesse dimensioni di quella in ingresso, risolvendo il problema della latenza intrinseca della pipeline.

Capitolo 6

FSM ed AXI-Stream

6.1 Implementazione della FSM e Interfacce AXI-Stream

Il cuore del sistema di filtraggio è costituito dalla logica di controllo, implementata attraverso una Macchina a Stati Finiti (FSM) che orchestra il riempimento dei *buffer line*, la generazione della finestra 3×3 e la gestione dei segnali del protocollo AXI4-Stream. In questo capitolo vengono analizzati nel dettaglio il diagramma degli stati, la gestione dei segnali di sincronizzazione (SOF ed EOL) e la logica di controllo del flusso dati (*backpressure*).

6.1.1 Architettura della FSM

La FSM ha il compito di coordinare tre segnali di controllo fondamentali per il funzionamento della pipeline:

- **pipeline_en**: segnale di abilitazione globale che determina l'avanzamento dei dati nei registri a scorrimento;
- **window_valid**: flag che indica se la finestra di convoluzione attuale contiene dati validi per il calcolo;
- **flush_pipeline**: segnale che gestisce la fase terminale dell'elaborazione, forzando l'ingresso a zero quando non vi sono più pixel in arrivo ma la pipeline deve ancora svuotarsi.

A differenza di implementazioni classiche, lo stato di **IDLE** è stato rimosso dalla logica esplicita degli stati per ottimizzare la risposta al segnale di *Start Of Frame* (SOF). La macchina opera principalmente su tre stati:

1. **LOADING**: In questa fase iniziale, il sistema riceve i dati in ingresso e riempie i *line buffer*. La finestra di convoluzione non è ancora valida e l'uscita è disabilitata.
2. **RUNNING**: Una volta riempita la finestra, il sistema entra in regime di elaborazione continua. Per ogni colpo di clock (se abilitato), viene prodotto un pixel filtrato in uscita.
3. **FLUSH**: Alla ricezione dell'ultimo pixel dell'immagine, la macchina entra in fase di svuotamento. L'ingresso della pipeline viene alimentato con valori nulli (zero padding) per permettere l'elaborazione dei pixel ai bordi finali dell'immagine. Completato lo svuotamento, la FSM torna in attesa.

6.1.2 Gestione del Protocollo AXI4-Stream

Il modulo agisce come un nodo di elaborazione trasparente all'interno della catena video, comportandosi come *Slave* sull'interfaccia di ingresso e come *Master* su quella di uscita.

Interfaccia Slave (Ingresso)

L'accettazione dei dati è governata dal segnale `s_axis_tready`. Questo viene asserito alto, segnalando la disponibilità a ricevere dati, a condizione che la FSM non si trovi nello stato di **FLUSH** e che il modulo a valle non stia esercitando *backpressure*, ovvero la porta in uscita non è pronta a ricevere dati. I segnali di controllo del frame vengono gestiti come segue:

- **SOF (Start Of Frame - tuser)**: Viene campionato per resettare istantaneamente la FSM e i contatori interni, garantendo l'allineamento all'inizio di ogni nuova immagine.
- **EOL (End Of Line - tlast)**: Utilizzato per il conteggio delle righe e per innescare la transizione verso lo stato di **FLUSH** al termine del caricamento dell'ultimo pixel.

Interfaccia Master (Uscita)

In uscita, il segnale di validità `m_axis_tvalid` riflette lo stato interno del segnale `window_valid`.

- Il segnale `m_axis_tuser` viene asserito per un singolo ciclo di clock in corrispondenza del primo pixel valido uscente.
- Il segnale `m_axis_tlast` viene generato internamente da un contatore di colonne che traccia la posizione del pixel corrente all'interno della riga elaborata.

6.1.3 Sincronizzazione e Rimozione dello stato IDLE

Per massimizzare l'efficienza e la reattività, la logica di reset e avvio è basata sul segnale `s_axis_tuser`. Invece di attendere in uno stato inattivo, la FSM monitora costantemente l'arrivo di un nuovo frame. Come mostrato nel Listing 6.1, l'arrivo di un `tuser` alto forza la macchina nello stato di `LOADING` e azzeri i contatori di latenza, indipendentemente dallo stato attuale.

```
1  -- Processo Sincrono della FSM
2  if rising_edge(s_axis_clk) then
3      if s_axis_rstn = '0' then
4          current_state <= LOADING;
5
6      elsif pipeline_en_s = '1' then
7          -- LOGICA DI SINCRONIZZAZIONE SOF
8          -- Un TUSER valido indica SEMPRE l'inizio di un nuovo
9          frame
10         if s_axis_tvalid = '1' and s_axis_tuser = '1' then
11             current_state <= LOADING;
12             latency_counter <= 1; -- Inizializzazione contatore
13             flush_pipeline_s <= '0'; -- Interruzione flush
14             precedenti
15             m_axis_tuser_s <= '0';
16         else
17             -- Evoluzione normale degli stati (LOADING, RUNNING,
18             FLUSH)
19             -- ...
20         end if;
21     end if;
22 end if;
```

Listing 6.1: Gestione sincrona del SOF e reset implicito

6.1.4 Controllo del Dataflow e Backpressure

La robustezza del filtro digitale in un contesto AXI-Stream dipende dalla corretta gestione della *backpressure*. Il sistema deve essere in grado di "congelare" l'intera pipeline se il ricevitore a valle non è pronto ad accettare dati (`m_axis_tready = '0'`).

La logica combinatoria che genera il segnale di abilitazione `pipeline_en_s` è descritta dalla seguente espressione VHDL:

```
1  pipeline_en_s <= '1' when ((s_axis_tvalid = '1' or
2      current_state = FLUSH)
3      and (m_axis_tready = '1' or window_valid_s = '0'))
4      else '0';
5
6  s_axis_tready <= '1' when (current_state /= FLUSH and
    m_axis_tready = '1')
    else '0';
```

Listing 6.2: Logica di abilitazione pipeline e gestione backpressure

Analizzando il codice nel Listing 6.2, l'abilitazione avviene solo se:

1. Vi è un dato valido in ingresso o la macchina è in fase di svuotamento (sorgente dati interna);
2. **E** il ricevitore è pronto (`m_axis_tready = '1'`) o l'uscita attuale non è valida (quindi non c'è rischio di perdere dati).

Se `m_axis_tready` scende a livello logico basso mentre è presente un dato valido in uscita, il segnale `pipeline_en_s` viene deassertito. Questo causa l'arresto immediato di tutti i registri a scorrimento e dei buffer, preservando lo stato interno del filtro fino a quando la congestione a valle non viene risolta.

6.2 Verifica Funzionale: Integrazione FSM e Buffer Line

Al fine di validare l'interazione tra la logica di controllo (FSM) e la pipeline di memoria (*Buffer Line*), è stata sviluppata una testbench specifica. L'obiettivo principale è verificare la correttezza del protocollo AXI-Stream (gestione dell'handshake) e la coerenza dei dati in uscita durante le fasi critiche di *Start of Frame* e *Flush*.

6.2.1 Setup di Simulazione

Per facilitare l'analisi visiva delle waveform e il debug, i parametri del design sono stati ridotti rispetto alla configurazione di sintesi finale. Il setup di simulazione prevede:

- **Dimensione Immagine:** Ridotta a 5×5 pixel (rispetto ai 32×32 o superiori del target reale). Questo permette di osservare un intero frame in un intervallo di tempo limitato.
- **Pattern Dati:** Generazione di un gradiente incrementale prevedibile. Il valore del pixel è calcolato come:

$$P(row, col) = (row \times N_{col}) + col$$

In questo modo, la sequenza di ingresso è composta da valori progressivi (0, 1, 2, ..., 24), rendendo immediata l'identificazione di eventuali disallineamenti o perdite di dati nella pipeline.

- **Obiettivo:** La simulazione si concentra sui casi limite (corner cases): l'allineamento iniziale, la gestione della backpressure e il riempimento automatico (padding) durante la fase finale di flush.

6.2.2 Generazione degli Stimoli e Handshake AXI

Il processo di stimolo della testbench emula una sorgente AXI-Stream Master. Un aspetto cruciale della verifica è la simulazione della *backpressure*: il testbench non si limita a inviare dati, ma rispetta rigorosamente i segnali di controllo provenienti dal DUT (Device Under Test).

Nel Listing 6.3 è riportato lo snippet VHDL che gestisce la generazione del frame. È fondamentale notare il loop di attesa su `s_axis_tready`: se la FSM abbassa il segnale di pronto (ad esempio durante un reset o una stallo interno), il generatore di stimoli sospende l'invio, garantendo la conformità al protocollo.

```
1  -- Processo di Stimolo: Generazione Immagine 5x5
2  process
3  begin
4  -- Attesa reset...
5  wait until rising_edge(clk);
```

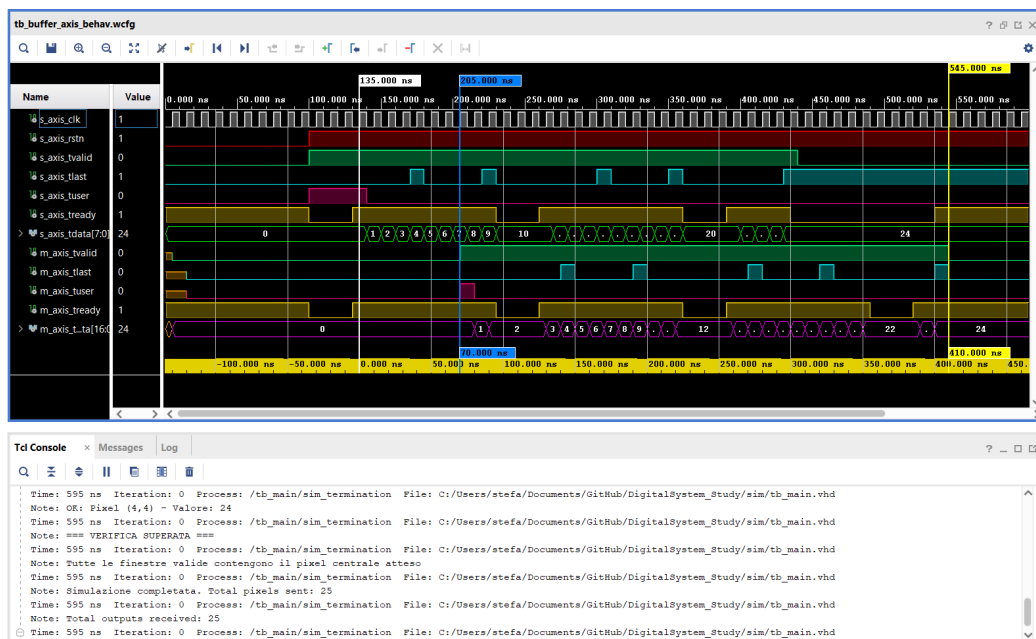


Figura 6.1: Waveform della simulazione: interazione tra FSM e Buffer Line durante la fase di Flush.

```

6
7  for row in 0 to NROW-1 loop
8  for col in 0 to NCOL-1 loop
9  -- Generazione Dato: Indice univoco per tracciabilita'
10 s_axis_tdata <= std_logic_vector(to_unsigned(row * NCOL +
    col, 8));
11 s_axis_tvalid <= '1';
12
13 -- Gestione Handshake (Backpressure Compliance)
14 -- Il dato rimane valido finche' la FSM non e' pronta (
    tready = '1')
15 loop
16 wait until rising_edge(clk);
17 exit when s_axis_tready = '1';
18 end loop;
19
20 end loop;
21 end loop;
22
23 -- Fine Frame: Disabilitazione validita' e attesa
    completamento flush
24 s_axis_tvalid <= '0';
25 wait for 100 ns;
26 -- ...
27 end process;

```

Listing 6.3: Processo di generazione stimoli con gestione Handshake AXI

6.2.3 Esiti della Verifica

La simulazione ha confermato i seguenti comportamenti chiave:

1. **Robustezza all'Handshake:** Il sistema gestisce correttamente le pause nel flusso dati. Quando `s_axis_tvalid` viene deassertito o quando la FSM deassertisce `s_axis_tready`, nessun dato viene perso o duplicato.
2. **Automatismo del Flush:** Al termine dei loop di generazione (quando l'ultimo pixel valido entra nel sistema), la FSM rileva correttamente la condizione di fine frame. Come visibile in Figura 6.1, il segnale `flush_pipeline` viene attivato internamente, forzando l'ingresso dei buffer a zero (padding). Questo permette alla finestra 3×3 di scorrere correttamente fino all'ultimo pixel dell'immagine, garantendo che l'output abbia le stesse dimensioni dell'input.